Summary and Foundations of x86-64 Assembly (Intel Syntax)

NECKER

November 14, 2025

Contents

1 Introduction: Intel vs. AT&T Syntax

Assembly language is a low-level programming language that corresponds almost directly to machine code. The **x86** architecture (and subsequent **x86-64**) supports two main syntaxes: **Intel** and **AT&T**.

1.1 Operand Convention (Intel Syntax)

The Intel syntax is characterized by the order of operands, which follows the logic:

$$\text{Instruction} \quad \text{Destination}, \text{Source}$$

The operation is conceptually an assignment: the content of the **Source** is moved into the **Destination**.

```
1    ; Intel: dest, source
2    mov rax, rbx    ; Copies the content of RBX into RAX
3    mov rax, [rcx]  ; Copies the value from the memory pointed to
     by RCX into RAX
4
```

Listing 1: Example of Intel Syntax

2 Register Types and Usage Conventions (x86-64)

The x86-64 architecture extends registers to 64 bits (prefix 'R') and increases their number. The use of registers follows precise **calling conventions** established by the operating system (e.g., System V ABI for Linux, or Microsoft x64).

2.1 General Purpose Registers (GPR)

Registers are 64 bits wide, but can also be accessed in smaller sections (EAX for 32 bits, AX for 16 bits, AL/AH for 8 bits).

- **RAX (Accumulator):** Used for arithmetic and function **return** values. In **syscalls**, it contains the number of the system function.
- **RBX (Base): Callee-saved** register (the called function must preserve it on the Stack). General purpose use.
- **RCX, RDX, R8, R9:** Primarily used to pass the first four **arguments** to functions (the precise order varies between Linux/Windows). In **syscalls**, they act as arguments.
- **RSI (Source Index):** Pointer to the source for operations on strings/blocks.

- **RDI (Destination Index):** Pointer to the destination for operations on strings/blocks. The first **argument** in System V ABI calling conventions (Linux).

2.2 Stack Control Registers

- **RSP (Stack Pointer):** Always points to the current **top** of the Stack. It is **mobile** and used by `PUSH` / `POP`.
- **RBP (Base Pointer):** Used as a **Frame Pointer**. It indicates the **fixed** start of the Activation Record (Stack Frame) of a function, facilitating access to local variables and arguments. It is *callee-saved*.

3 Assembler Directives for Space Reservation

Directives are instructions for the assembler (not for the CPU) used to define sections, allocate variables, and reserve space in memory, particularly in the data segment (`.data`) or the Stack segment (`.stack`).

3.1 Section Directives

- `.data` or `section .data`: Defines the segment in which initialized variables and data are allocated, meaning those already defined at the start of the program (predefined output strings).
- `.bss` or `section .bss`: Defines the segment for **uninitialized** variables and data (variables that will exist).
- `.text` or `section .text`: Defines the segment for executable code (instructions).

3.2 Data Definition Directives

These directives reserve space and, optionally, initialize it. The general format is: `Variable_Name Directive Value(s)`.

Directive	Full Name	Size
`DB`	Define Byte	1 Byte
`DW`	Define Word	2 Bytes (16 bits)
`DD`	Define Doubleword	4 Bytes (32 bits)
`DQ`	Define Quadword	8 Bytes (64 bits)

3.2.1 Space Reservation (Uninitialized)

To reserve space without initializing (typically in `.bss`), directives such as `RESB`, `RESW`, `RESD`, `RESQ` are used, with the argument indicating the **number of units** to reserve.

```
1    section .data
2    msg DB 'Hello', 0   ; 'msg' variable: 6 initialized bytes (
     string + null terminator)
3
4    section .bss
5    buffer RESB 1024    ; 'buffer' variable: Reserves 1024 bytes
6    contatore RESQ 1    ; 'contatore' variable: Reserves 8 bytes (a
      quadword)
7
```

Listing 2: Examples of Space Reservation

4 Fundamental Assembly Instructions

Instructions are the commands executed by the CPU.

4.1 Data Movement

- `MOV` *`dest, src`*: Moves data between registers, between register and memory, or between an immediate value and a register/memory.
- `PUSH` *`src`*: Moves the operand onto the Stack and decrements `RSP`.
- `POP` *`dest`*: Extracts the operand from the top of the Stack and increments `RSP`.

4.2 Arithmetic and Logic

- `ADD` *`dest, src`*: $dest = dest + src$.
- `SUB` *`dest, src`*: $dest = dest - src$.
- `MUL` *`src`*: Multiplies *src* by the **RAX** register. The 128-bit result is stored in **RDX:RAX**.
- `INC` *`dest`*: $dest = dest + 1$.
- `DEC` *`dest`*: $dest = dest - 1$.
- `AND, OR, XOR, NOT`: Bitwise logical operations.

4.3 Flow Control

- `CMP` *`op1, op2`*: Compares operands (*op1* - *op2*) and sets status flags (ZF, CF, SF) without modifying the operands.
- `JMP` *`label`*: Unconditional jump to the label.
- `J`*`cond label`*: Conditional jump based on flags (e.g., `JE` for jump if equal/Zero Flag set, `JG` for jump if greater, etc.).

- `CALL` *`label`*: Saves the return address on the Stack and performs a jump.
- `RET`: Extracts the return address from the Stack and jumps to that address.

5 Interfacing with the Operating System (Syscalls)

System Calls (`syscall`) are the mechanism through which an Assembly program requests services from the operating system kernel (e.g., I/O, process creation, memory allocation).

5.1 Linux/System V x86-64 Convention

The execution of a `syscall` follows these steps (Linux):

1. Load the **syscall number** into the **RAX** register.
2. Load the **arguments** of the syscall into specific registers:
   - Argument 1: **RDI**
   - Argument 2: **RSI**
   - Argument 3: **RDX**
   - Argument 4: **R10**
   - Argument 5: **R8**
   - Argument 6: **R9**
3. Execute the `syscall` instruction.
4. The **return** value of the syscall is written into **RAX**.

5.2 Example: `exit` Syscall (exiting the program)

```
1    section .text
2    global _start
3
4    _start:
5    ; 1. Syscall code (exit = 60) in RAX
6    mov rax, 60
7    ; 2. Argument 1 (exit code) in RDI
8    mov rdi, 0      ; Exit code 0 (success)
9    ; 3. Executes the syscall (which is contained in the RAX
     register)
10   syscall         ; Terminates the program
11
```

Listing 3: Example of a Syscall to Exit (exit)